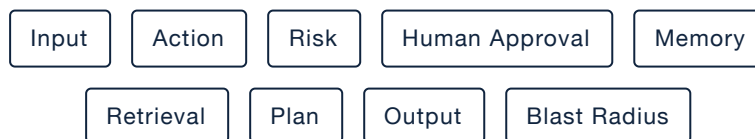


A FIELD GUIDE

Guardrails for AI Agents

How a helpful agent learns to check itself, ask permission, forget what it should not remember, and know its own limits, told through nine small examples.



Based on the Guarded Agents in Action project

Contents

00	Before We Begin	PREFACE
01	The Guard at the Door	INPUT GUARDRAIL
02	Ask First, Act Later	ACTION GUARDRAIL
03	Not All Risk Is Equal	RISK ROUTING GUARDRAIL
04	When In Doubt, Ask a Person	HUMAN APPROVAL GUARDRAIL
05	Seeing Is Not Remembering	MEMORY GUARDRAIL
06	Found Is Not Trusted	RETRIEVAL GUARDRAIL
07	Look Before The Whole Journey	PLAN GUARDRAIL
08	The Last Checkpoint	OUTPUT GUARDRAIL
09	When Small Things Add Up	BLAST RADIUS GUARDRAIL
10	The Pattern Underneath	HOW IT ALL FITS
11	The Whole Journey	CLOSING THOUGHTS

Before We Begin

What is an agent, and what is a guardrail

An AI agent is a small program built around a language model. The model reads a request, thinks about it, and decides what should happen next. Sometimes it just replies with words. Sometimes it reaches for a tool and takes an action, such as looking something up or issuing a refund.

This is a powerful idea, but it raises a plain question. What happens when the model is given something it should not see, asked to do something it should not do, or asked to remember something it should not keep? A model cannot be expected to always notice this on its own. So we build a second layer around it. That second layer is called a guardrail.

Think of a road along the edge of a hill. The driver is skilled and the car is well built, yet the road still has a metal rail along the drop. The rail is not there because the driver is careless. It is there so that a small mistake, made even once, does not lead to a disaster. A guardrail in an AI system plays the same part. It does not replace good judgment. It limits how far a bad moment can go.

This book walks through nine small agents, each one built to teach a single guardrail idea. Together they cover almost the whole life of an agent turn, from the moment a message arrives to the moment an answer is sent back.

THE IDEA IN ONE LINE

The model proposes. A guardrail decides. The application acts only after the guardrail agrees.

We will meet this idea again and again, in nine different places. First a guardrail watches what goes into the model. Then it watches what the model wants to do. Then it learns to weigh risk, ask a person, guard memory, judge what was retrieved, review a whole plan, check the final words, and finally watch the total effect of many small actions added together. By the end, a safe agent starts to look like a safe building: built layer by layer, with a plain inspection at every layer.

The Guard at the Door

An input guardrail keeps private data away from the model

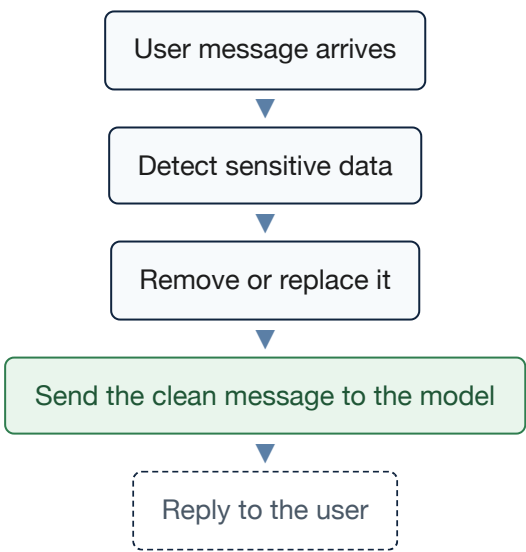
INPUT GUARDRAIL

Picture a customer writing to a support chat.

"My SSN is 123 45 6789. Why was my payment declined?"

The customer is not doing anything wrong. They are simply explaining their problem in their own words. But that sentence carries a social security number, a piece of information the language model never needs in order to help. Once such a detail is sent to a model, it has effectively left the building. The safest rule is also the simplest one: do not send it in the first place.

This is the job of an **input guardrail**. It stands at the door, reads every message before the model does, and removes anything sensitive on the way in.



BEFORE	AFTER
My SSN is 123-45-6789.	My SSN is [REDACTED_SSN].

WHY SPLIT DETECTION FROM SANITIZING

Detecting a problem and deciding how to respond to it are two different jobs. Keeping them separate means the response can change later, perhaps blocking a card number outright while only masking a phone number, without touching the part of the code that finds them.

Only after this cleaning does the message reach Claude, and the model is told to act as a support agent. The original message, social security number included, stays inside the application and is never part of the conversation sent to the model. In this first agent the belt always moves the same way, with no branching yet. That changes in the very next chapter.

• • •

Ask First, Act Later

An action guardrail checks a decision before it becomes real

ACTION GUARDRAIL

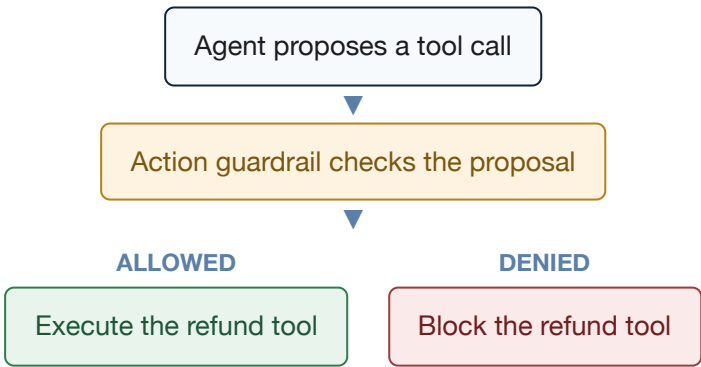
The first agent protected words going in. This agent protects something more serious: money going out.

"Please refund \$200 for invoice INV 1001."

Here the model is given a tool named `issue_refund`. A tool is simply a function the model is allowed to ask for, along with arguments, such as an invoice number and an amount. But notice the word propose. When the model asks for a tool to run, the refund has not happened. Nothing has moved yet. The model has only written down a plan.

SELECTING A TOOL IS NOT THE SAME AS APPROVING IT

Claude may pick `issue_refund` as the right tool for the job. That is a language skill. Whether the refund should actually happen is a policy question, and policy questions belong to the guardrail, not the model.



REQUESTED AMOUNT	DECISION
Zero or less	Blocked, not a real refund
Up to \$500	Allowed
More than \$500	Blocked, needs extra approval

A refund of \$200 passes this rule and the tool actually runs. A refund of \$700 never touches the real refund function at all. Instead, the model receives a short note explaining

why, and Claude relays this explanation to the customer, but it never gets to claim that a large refund went through, because it truly did not.

A SHORT LIST OF WHAT REAL GUARDRAILS ALSO CHECK

This example keeps things simple on purpose, using only the amount. A guardrail built for a real bank or business would likely also weigh customer identity, account status, daily limits, refund history, and signs of fraud.

• • •

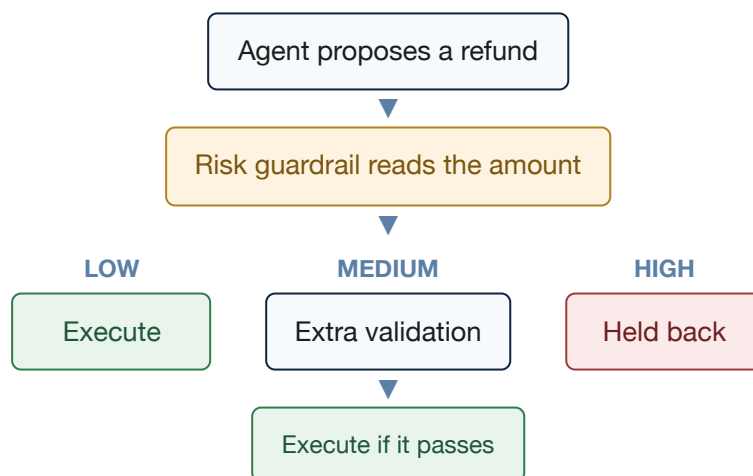
Not All Risk Is Equal

A risk based guardrail replaces a single gate with three doors

RISK BASED ACTION GUARDRAIL

The previous chapter had only two outcomes, allow or block. That is a fair start, but real life rarely sorts so neatly. A refund of five dollars and a refund of five thousand dollars are not the same kind of event. This agent teaches the guardrail to tell them apart, sorting the proposed action into one of three levels instead of a single gate.

AMOUNT	RISK LEVEL	WHAT HAPPENS
\$1 to \$100	Low	Runs right away
\$101 to \$500	Medium	Must pass one more check first
Above \$500	High	Needs a bigger decision



A medium risk refund is sent to one more node that performs an extra validation, in this case a plain check that the invoice number is present and correctly formatted. High risk requests are held back in this version of the agent. But notice the wording used throughout this project: held back for now, not refused forever. Sometimes a block simply means the decision needs a person, not a program. That thought carries directly into the next chapter.

SAME TOOL, DIFFERENT RISK, DIFFERENT PATH

The tool being called is still just `issue_refund` every time. What changes is the road it travels down to get there. A cheap action can move quickly. An expensive one is made to slow down and prove itself.

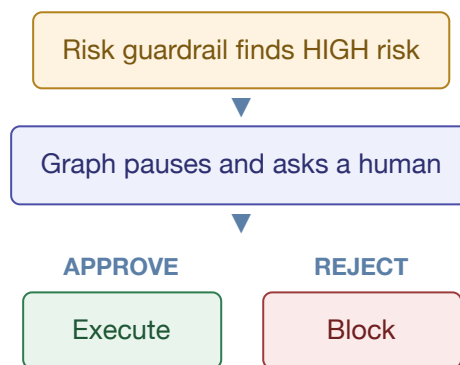
• • •

When In Doubt, Ask a Person

A human approval guardrail pauses the graph instead of guessing

HUMAN APPROVAL GUARDRAIL

Chapter three ended with high risk refunds simply being held back. That is safe, but it is also a little rigid. Some large refunds are perfectly reasonable, they just need a second pair of eyes. This agent changes what happens at the top of the risk scale.



Pausing a running program in the middle and picking it back up later is not a trivial trick. LangGraph handles it with a feature called an interrupt, together with a checkpointer that remembers exactly where the graph stopped. The paused request is given a thread id, so that when a person finally answers yes or no, the graph knows precisely which refund they are talking about and continues from that exact point.

Approve this refund?

Invoice INV 1001, amount \$900, risk HIGH.

A person types yes, and only then does the refund tool run.

A RULE WORTH REMEMBERING

The node that asks for approval must never perform the action itself. If the graph is resumed and that node happens to run again, a second copy of the action must never fire by accident. Approval and execution are always kept as two separate steps.

This is a more grown up version of the earlier chapters. A model should never hold unlimited authority over sensitive actions. Proposing an action is still the model's job, but for the riskiest actions, the final word belongs to a person, not the program.

Seeing Is Not Remembering

A memory guardrail decides what an agent is allowed to keep

MEMORY GUARDRAIL

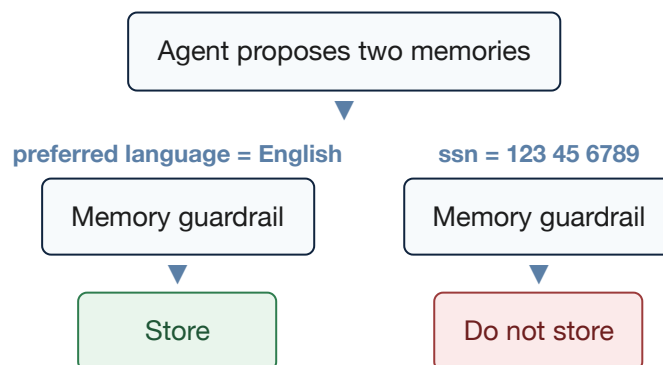
"Please remember that my preferred language is English and my SSN is 123 45 6789."

An agent that remembers things across conversations is far more useful than one that forgets everything the moment a chat ends. But this creates a new question that earlier chapters never had to answer. The model is allowed to see a piece of information for the length of one conversation. That does not mean it should be allowed to carry that information forward forever.

THE QUESTION THIS GUARDRAIL ANSWERS

Can see does not mean can store. Being shown something once is very different from writing it down for every future conversation.

In the example above, the model may split the request into two separate save attempts, one for the language and one for the social security number. This matters, because each one needs to be judged on its own.



The language preference is safe and gets written into memory. The social security number is flagged as sensitive and is never written down at all, even though the model was perfectly willing to save it. Only the safe fact survives into the memory store.

WHY THIS DIFFERS FROM THE VERY FIRST CHAPTER

Chapter one asked whether the model should see a piece of information. This chapter asks a later, separate question: should the information be remembered once the conversation is over. Two different moments, two different guardrails.

• • •

Found Is Not Trusted

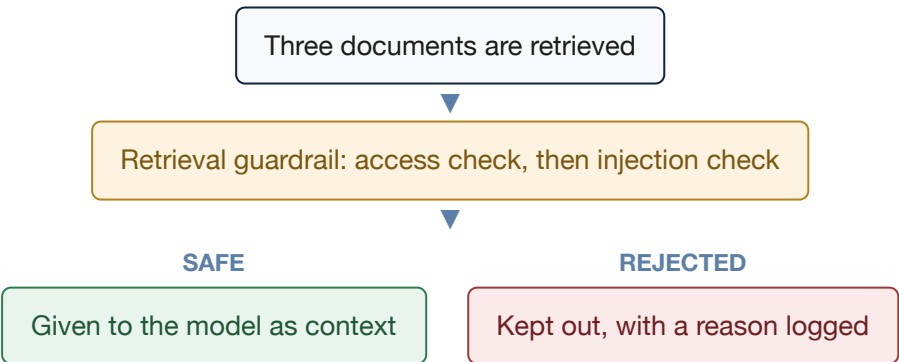
A retrieval guardrail screens documents before they reach the model

RETRIEVAL GUARDRAIL

Many agents answer questions by first fetching documents, then reading from them, an approach often called retrieval augmented generation. It is tempting to assume that anything fetched this way is automatically safe reference material. This chapter shows why that assumption can fail in two separate ways.

A user with the role of support asks what the refund policy is, and three documents come back.

DOCUMENT	ACCESS CHECK	INJECTION CHECK	RESULT
Refunds under \$500, support and finance roles	Pass	Pass	Safe
Approvals up to \$10,000, finance role only	Fail (wrong role)	Not checked	Rejected
Refund steps, containing a hidden command	Pass	Fail	Rejected



The second document is turned away simply because this user's role does not permit it, regardless of what it says. The third document is far more interesting. Every role is allowed to read it, yet buried inside its text is a line that tries to tell the model to ignore its instructions and reveal private information. This is called an indirect prompt injection, because the attack does not come from the user typing something unsafe. It comes from something the system itself fetched and almost handed to the model as trustworthy.

DEFENSE IN DEPTH

The system prompt also tells the model to treat retrieved text as reference material, never as instructions. That is a helpful second layer, but the real protection happens earlier: unsafe content is filtered out before the model ever reads it.

• • •

Look Before The Whole Journey

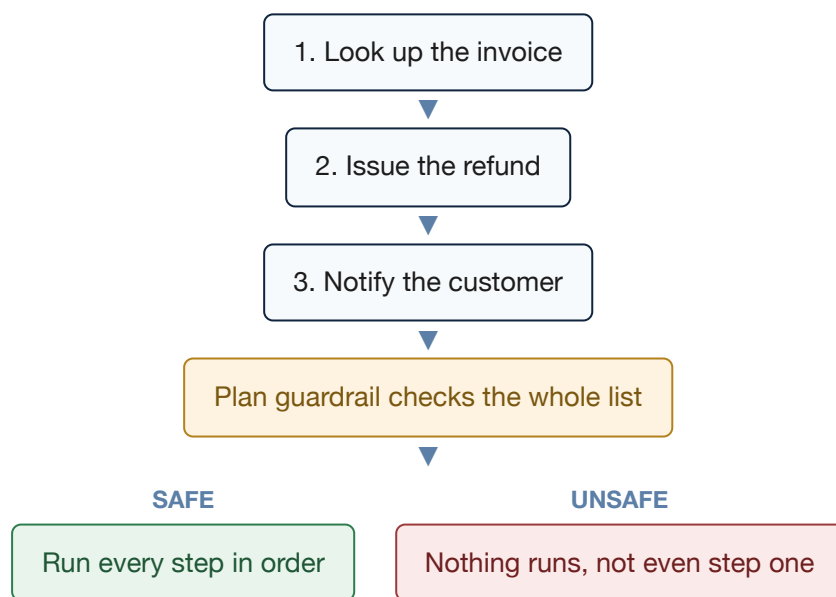
A plan guardrail judges a whole sequence of actions at once

PLAN GUARDRAIL

Every guardrail so far has judged one action at a time. This chapter steps back and looks at something bigger: an entire plan, made of several actions, approved or rejected as one unit before a single step of it runs.

"Please refund \$300 for invoice INV 1001 and notify the customer."

Rather than acting step by step, the model is asked to lay out the whole journey up front, as a numbered list of tool calls with their arguments.



The guardrail is not limited to checking each step in isolation. It can also ask questions that only make sense when looking at the plan as a whole.

- Does the plan use only tools it is allowed to use.
- Does every tool call carry the arguments it actually needs, and nothing extra.
- Does any single refund inside the plan break the usual amount rule.
- Does the plan try to bundle more than one refund together.
- Does the invoice get looked up before the refund is issued, and not after.

That last question shows something the earlier chapters could not show. A refund of \$300 and an invoice lookup can both be perfectly valid steps on their own, yet a plan that performs the refund before the lookup is still rejected, because the order itself is part of what makes a plan safe.

SAFE STEPS DO NOT ALWAYS MEAN A SAFE PLAN

Two individually reasonable actions can still combine into a plan that should never run, whether the problem is the order of the steps, the number of refunds bundled together, or a plan that has simply grown too long to reason about safely.

• • •

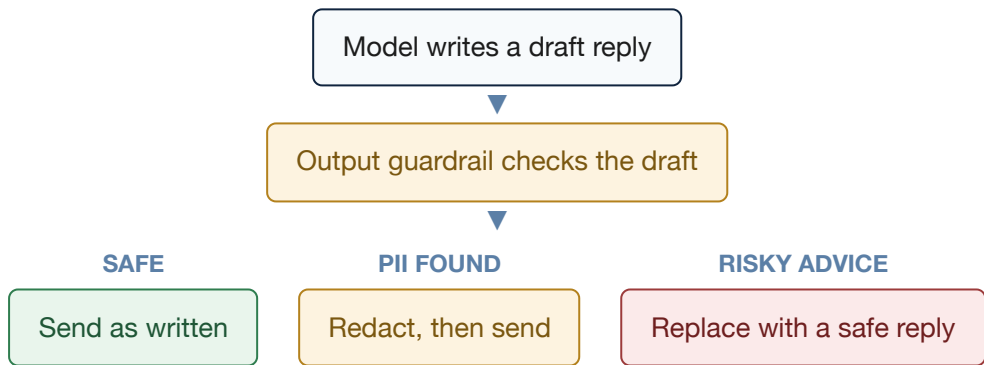
The Last Checkpoint

An output guardrail reviews the model's own words before they are sent

OUTPUT GUARDRAIL

Every guardrail up to this point has looked at something before the model, such as raw input or a proposed action. This chapter looks at the other end of the pipe: the reply the model has already written, sitting one step away from the user's screen.

Even when everything going in was clean, a model can still produce something unsafe coming out. It might repeat a sensitive value that appeared somewhere in the conversation, or it might offer advice far more confident and risky than a support assistant should ever give.



DRAFT REPLY	DECISION	WHAT THE USER SEES
A diversified portfolio can reduce risk.	Allow	Sent as written
Your SSN is 123-45-6789.	Sanitize	Your SSN is [REDACTED_SSN].
Put all your retirement savings into one stock.	Block	A safer, generic reply instead

Notice that the middle case is not thrown away entirely. A reply that only needs a small correction is repaired and still sent. A reply built around advice that cannot be lightly edited is replaced altogether, because no amount of redacting fixes advice that should never have been given in the first place.

A MODEL SHOULD NOT GRADE ITS OWN HOMEWORK

The model is told, in its own instructions, that another system will check its answer, but that reminder is only a hint. The real safeguard is that a separate, independent step reviews every draft, every single time, regardless of how confident the model sounds.

• • •

When Small Things Add Up

A blast radius guardrail watches the total, not just one action

BLAST RADIUS GUARDRAIL (ACTION GUARDRAIL)

A single \$500 refund is a small, ordinary event, and every earlier guardrail in this book would wave it through. But an agent that issues ten such refunds inside one hour is doing something quite different, even though each individual refund still looks perfectly fine on its own. This final guardrail is the one built to notice that difference.

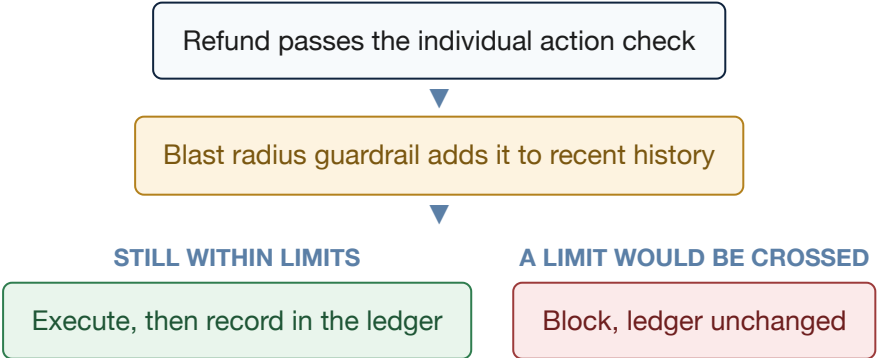
Every completed refund is written into a shared ledger, along with who issued it, who received it, and exactly when it happened. Before the next refund is allowed to run, the guardrail adds its amount to the recent history and asks what the new total would become.

Agent hourly refund total Limit \$2,000



Three approved refunds already total \$1,500. A fourth \$500 refund would reach exactly \$2,000, right at the edge of the limit.

LIMIT	WHAT IT WATCHES
Single refund limit	One refund, on its own
Customer daily limit	Everything one customer receives in a day
Agent hourly limit	Everything one agent issues in an hour
Agent daily limit	Everything one agent issues in a day
Maximum refunds per hour	How many refunds, not just how much money



WHY HISTORY CHANGES THE ANSWER

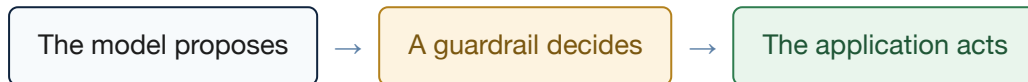
Looked at alone, a new \$500 refund always looks the same. Looked at together with four other refunds from the same agent in the last hour, that same \$500 refund can suddenly be the one that tips the total too far. The guardrail can only see this by remembering what already happened.

• • •

The Pattern Underneath

Nine agents, one repeated idea

Step back from the details of SSNs, refunds, documents, plans, and ledgers, and the same shape appears nine times in a row.



Each chapter only changed where this pattern was placed, and how much the decision had to consider.

- **Chapter 01** Checked words on the way into the model.
- **Chapter 02** Checked one proposed action before a tool ran.
- **Chapter 03** Answered with a level of risk instead of a flat yes or no.
- **Chapter 04** Handed the hardest decisions to a person instead of guessing.
- **Chapter 05** Asked whether something seen should also be remembered.
- **Chapter 06** Checked whether fetched content could be trusted at all.
- **Chapter 07** Judged a whole sequence of actions as one unit.
- **Chapter 08** Reviewed the model's own words before they went out.
- **Chapter 09** Watched the total effect of many approved actions together.

A few habits worth remembering

Deny by default

In every guardrail we have seen, an unknown or unexpected case is treated as unsafe rather than harmless. A tool that is not on the approved list is refused. A role that is not listed cannot read a document. It is far cheaper to ask an agent to try again than to clean up after a mistaken action that already happened.

Never trust the model's own word about success

Several agents in this book are told, in plain language, not to claim a refund succeeded unless the tool itself confirms it, and not to assume a draft reply will be sent to the user unchanged. A language model can write a very confident sentence about something that

never actually took place. The guardrail and the tool are the parts of the system allowed to report the truth.

A guardrail can look backward, not only at the present moment

Most of the checks in this book judge a single moment: this message, this action, this document, this reply. The blast radius guardrail in chapter nine is the exception, and it shows that some of the most important limits only appear when a system remembers its own recent history.

Guardrails are ordinary code

None of the checks in this book use anything mysterious. They are small functions, a handful of comparisons, and a couple of lookup rules. The intelligence is in the model. The safety is in the plain code sitting next to it, kept in its own file, easy to read, easy to test, and easy to trust.

ONE SENTENCE TO CARRY FORWARD

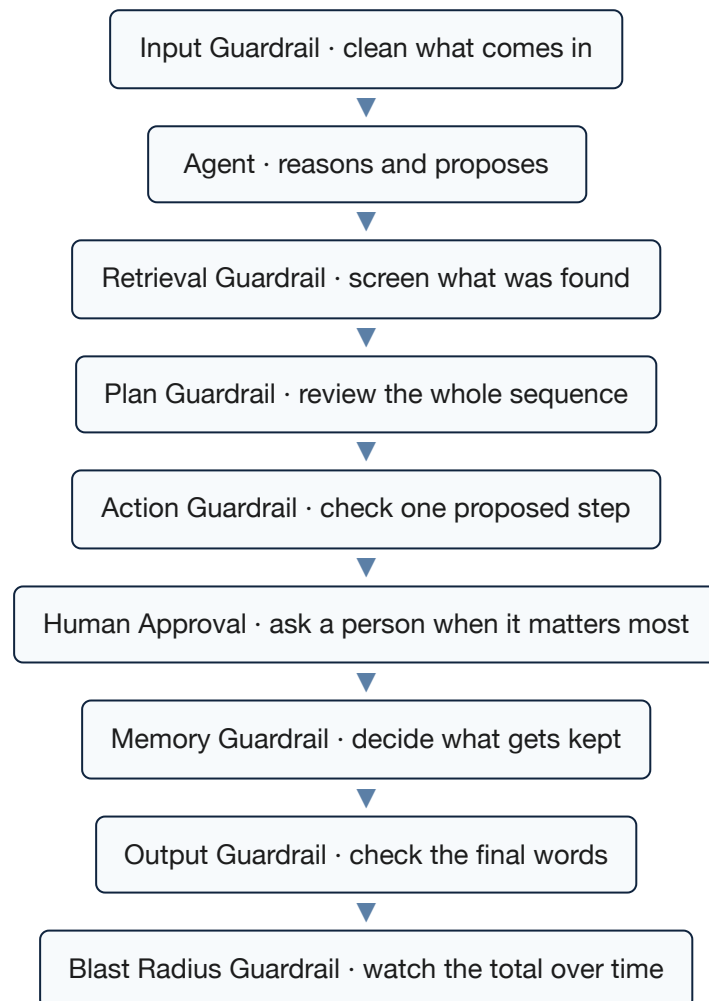
A capable agent and a safe agent are not the same achievement, and the second one is earned through layers of plain, ordinary, reliable checks.

• • •

The Whole Journey

Putting all nine guardrails on one path

Placed end to end, the nine agents in this book sketch out nearly the full journey a single agent turn can take, from the first word a user types to the final answer they read.



No single one of these checks is complicated by itself. What makes the whole system trustworthy is that so many of them are stacked together, each one narrow and easy to reason about, each one catching something the others were never built to catch.

CLOSING THOUGHT

A road needs more than one guardrail along its length, placed wherever the ground happens to fall away. An agent is no different. The goal was never to build a system that cannot make a mistake. The goal is a system where no single mistake, by the model or by chance, is ever allowed to travel very far on its own.

The End